Cérémonie d'implémentation

La cérémonie d'implémentation est probablement la plus connue, mais c'est aussi celle qu'on comprend le plus souvent de façon trop étroite. Ici, on n'est pas en mode discussion de besoin, ni en mode raffinement, ni en mode assemblage final. On est en mode **build**. On exécute ce qui a été préparé en amont, après la **Definition of Ready (DoR)**, dans un cadre contrôlé, traçable et discipliné.

Cette cérémonie s'applique autant en **sprint**, en **kanban** ou en mode **hybride**. Elle s'applique autant pour une **nouvelle feature**, une **amélioration d'une feature existante**, un **correctif** ou un lot mêlant plusieurs types de travail. Le mode de flux change, mais le principe reste le même : **on construit ce qui a été défini et planifié**.

> **Note terminologique.** Cette page décrit la cérémonie d'implémentation dans son contexte opérationnel. Pour les définitions de référence des termes comme implémenter, spécification, spécification atomique, DoR, DoD, validation, vérification, test, niveaux d'intégration, interfaces, angles, intégration, contrat d'intégration, CI/CD, artefact, dette technique, qualimétrie, rôles et RACI, voir le Lexique / Terminologies. Les contributions indiquées dans cette page sont propres à cette cérémonie et ne remplacent pas la matrice RACI.

Ici, **implémenter** ne veut pas dire assembler, boulonner, monter ou intégrer la livraison dans la vraie réalité opérationnelle. Ces dimensions deviennent centrales dans **DIFFUSER**. Cela ne veut pas dire qu'on ne fait pas de *continuous deployment* ou qu'on bloque les pipelines. Cela veut dire qu'à ce stade, le centre de gravité est le **build conforme**, pas la mise en circulation réelle.

🎯 Objectif

La cérémonie d'implémentation transforme les spécifications atomiques approuvées et les stories prêtes en **code, tests, validations et preuves de conformité**. C'est une des parties cœur de **FORGE** : l'endroit où l'on simule la réalité de manière suffisamment fidèle pour démontrer que ce qui a été demandé peut réellement fonctionner, avant sa diffusion réelle.

> Implémenter, ce n'est pas improviser une solution en direct. C'est exécuter avec rigueur un plan déjà préparé, tout en capturant les signaux qui révèlent un écart entre la réalité observée et ce qui avait été élicité, défini ou planifié.

🧱 Ce que veut dire « implémenter » ici

Dans cette cérémonie, implémenter signifie :

- **Coder** ce qui a été spécifié.
- **Valider** que le comportement développé correspond réellement aux spécifications.
- **Prouver** que le résultat obtenu concorde avec les **spécifications IF... THEN...**.
- **Maintenir la traçabilité** entre spécification, story, code, tests et résultats.

On reste donc dans une logique de **construction contrôlée**. Même si plusieurs validations d'intégration ont déjà lieu ici, elles servent d'abord à démontrer la conformité du build et la cohérence de la solution, pas encore à parler du montage final, de la diffusion progressive, du rollout, des mécanismes de réaction terrain ou de l'activation réelle dans l'écosystème complet.

👥 Participants recommandés

Rôle	Responsabilité principale
Développement	Construire conformément aux spécifications, produire les tests, valider la conformité et remonter les signaux d'écart.
QA / qualité	Valider que les **spécifications IF... THEN...** et les comportements attendus sont réellement démontrés à plusieurs niveaux.
PO / PM	Protéger le scope prévu, arbitrer les écarts et décider si un signal doit devenir un retour formel vers ÉLICITER.
Architecture / lead technique	Confirmer la cohérence technique, les contrats et la solidité des validations d'intégration nécessaires au build.
CdL / coordination	Maintenir la visibilité du flux, des risques, des écarts et des décisions de recadrage.

📥 Inputs / Prérequis

La cérémonie d'implémentation commence **après** la DoR. Cela veut dire que l'équipe n'est plus censée être en train de découvrir le besoin, d'inventer le scope ou de redéfinir le plan de match principal.

Avant IMPLÉMENTER	Ce qui doit déjà être vrai
ÉLICITER	Le besoin, la valeur, les signaux et les métriques ont été compris et documentés.
DÉFINIR	Les spécifications atomiques sont complètes, cohérentes, testables et approuvées.
PLANIFIER	Les stories sont découpées, traçables, priorisées et prêtes à être exécutées.
DoR	Les prérequis d'entrée sont satisfaits : clarté, testabilité, dépendances, stratégie de test, capacité et environnement.

⚠️ Si l'équipe commence à « découvrir » massivement pendant l'implémentation, c'est généralement le signe qu'une porte amont n'était pas réellement fermée.

📤 Outputs / Livrables attendus

À la fin de la cérémonie, on doit pouvoir démontrer que le build est conforme, testé, traçable et prêt à passer vers DIFFUSER.

- Du **code conforme** aux spécifications atomiques et aux stories ayant passé la DoR.
- Des **tests exécutés** et des résultats documentés aux niveaux pertinents : unitaires, contrats, intégration, non-régression, validations fonctionnelles ou techniques.
- Des **spécifications IF… THEN… démontrées** avec des preuves observables.
- Une **traçabilité maintenue** entre story, spécification, code, tests et résultats.
- Les **écarts, signaux, métriques ou problèmes** capturés et remontés aux phases appropriées lorsque nécessaire.
- Un incrément suffisamment prouvé pour entrer dans **DIFFUSER**.

⏱️ Durée, format et sous-phases recommandées

La durée de l'implémentation dépend du type de travail, de la complexité technique, du nombre d'intégrations touchées et du niveau de validation requis. L'objectif n'est pas de forcer une durée

uniforme, mais de rendre visible tout le travail réel nécessaire pour construire et prouver correctement l'incrément.

La cérémonie peut être vue en deux sous-phases complémentaires :

1. **Build et validations de base** — les développeurs construisent le comportement prévu, écrivent les tests nécessaires, valident localement et confirment que le code respecte les **spécifications IF... THEN...**.
2. **Validations croisées et intégration contrôlée** — l'équipe vérifie la concordance avec QA, architecture ou les personnes pertinentes, exécute les validations d'intégration nécessaires, réalise les revues et documente les preuves.

Durées indicatives :

- **Bug simple ou correctif ciblé :** quelques heures à 1 journée, selon la facilité de reproduction, les tests requis et le risque de régression.
- **Story simple :** 1 à 3 jours pour construire, tester, réviser et documenter les preuves.
- **FEATURE complexe ou sujet transversal :** plusieurs jours ou plusieurs itérations peuvent être nécessaires, surtout si plusieurs composants, contrats ou équipes sont touchés.

Une FEATURE peut rester longtemps en IMPLÉMENTÉ. C'est normal lorsque le build exige plusieurs validations, lorsque les intégrations sont nombreuses, lorsque des environnements doivent être stabilisés ou lorsque des preuves supplémentaires sont nécessaires avant DIFFUSER. Ce temps doit être comptabilisé comme du vrai travail d'équipe.

🔄 Déroulement suggéré

- **Confirmer la DoR, le contexte et les signaux/métriques pertinents des phases amont** — vérifier que la story ou le lot est prêt, traçable, relié aux spécifications approuvées et ancré dans les signaux, métriques et risques identifiés en ÉLICITER, DÉFINIR et PLANIFIER.

1. **Reprendre le plan de match** — clarifier la séquence d'implémentation, les dépendances, les validations attendues et les critères de succès.
2. **Implémenter le code** — construire strictement le comportement défini, sans élargir le scope de façon opportuniste.
3. **Écrire et exécuter les tests** — couvrir les **spécifications IF... THEN...**, les cas limites prévus, les contrats et les risques de régression.
4. **Valider la concordance** — vérifier que code, validations, **spécifications IF... THEN...** et story racontent la même histoire.
5. **Réaliser les validations d'intégration pertinentes** — simuler ou contrôler les interactions nécessaires pour prouver que le build tient.

6. **Effectuer les revues** — revue de code, revue QA, validation technique ou validation produit selon le contexte.
7. **Capturer les écarts** — documenter les signaux, métriques, problèmes ou nouvelles hypothèses découvertes pendant l'exécution.
8. **Confirmer la sortie vers DIFFUSER** — s'assurer que l'incrément est suffisamment prouvé, traçable et stable pour passer à la phase suivante.

🔀 Sprint, Kanban ou hybride

La cérémonie fonctionne dans plusieurs modes opératoires. Ce qui change, c'est la manière de piloter le flux, pas la nature du travail d'implémentation.

Mode	Lecture de la cérémonie	Point de vigilance
Sprint	Le lot de stories est engagé pour un horizon court, avec un objectif de sprint explicite.	Ne pas transformer le sprint en espace d'improvisation continue sur le scope.
Kanban	Les stories prêtes sont tirées selon la priorité et les limites WIP, dans un flux continu.	Ne pas utiliser le flux continu comme prétexte pour modifier constamment la cible.
Hybride	Certaines activités suivent un engagement temporel, d'autres un réapprovisionnement continu.	Clarifier ce qui est engagé, ce qui est tiré et ce qui doit repasser par les phases amont.

Autrement dit, en sprint comme en kanban, l'équipe peut rester agile, mais **à un degré plus mince rendu ici**. Cette réduction volontaire de flexibilité n'est pas une perte d'agilité : c'est la conséquence normale du fait que l'agilité devait déjà s'exprimer fortement en amont, dans l'élicitation, la définition et la planification.

🧩 Types de travail couverts

La cérémonie d'implémentation couvre plusieurs natures de travail, sans changer son principe fondamental.

- **Nouvelle feature** : construire un comportement nouveau à partir de spécifications approuvées.
- **Amélioration d'une feature existante** : faire évoluer un comportement existant de manière contrôlée.

- **Correctif** : corriger un écart ou une défaillance déjà compris et correctement recadrés en amont.
- **Hybride** : livrer un ensemble combinant nouveaux comportements, ajustements et correction de régressions.

Dans tous les cas, le réflexe doit rester le même : **on bâtit ce qui a été préparé**, on ne transforme pas l'implémentation en espace de redécouverte sans fin.

🧪 Une simulation fidèle de la réalité

Cette cérémonie fait partie du cœur de FORGE parce qu'elle cherche à **simuler la réalité sans être encore la vraie réalité de diffusion**. On veut vérifier que la solution tient, que les comportements sont conformes, que les tests passent, que les interactions prévues fonctionnent et que les validations d'intégration nécessaires confirment la cohérence du tout.

Il y a donc ici **beaucoup de validations**, y compris à différents niveaux d'intégration :

- validation du comportement local ;
- validation des contrats ;
- validation entre composants ;
- validation des flux critiques ;
- validation de la non-régression ;
- validation de la conformité aux spécifications.

Mais même si ces validations existent déjà, on ne doit pas confondre cette phase avec la phase de diffusion réelle. Ici, on veut démontrer que le build est solide. Dans DIFFUSER, on parlera davantage d'assemblage final, d'activation réelle, de circulation contrôlée, d'exposition, de réaction et d'observation en conditions réelles.

📏 Flexibilité oui, improvisation non

Une fois en implémentation, l'équipe doit rester professionnelle, lucide et capable de s'adapter. Mais elle ne doit pas tomber dans un mode où l'on ajoute, change ou améliore en direct simplement parce que « tant qu'on y est » une autre idée apparaît.

> Si on commence à ajouter live, changer d'idée au fil de l'eau, ou améliorer de façon improvisée, on détruit rapidement la concordance entre l'élicitation, la définition, la planification et l'exécution.

Le danger est de dériver vers une version de travail type **Dropbox** :

- on ajoute des petites choses non prévues ;

- on change la cible pendant qu'on code ;
- on améliore en direct sans nouvelle validation produit ;
- on garde les modifications « parce qu'elles semblent bonnes » ;
- on finit par ne plus savoir ce qui était demandé au départ.

Quand cela arrive, la définition ne concorde plus, le plan de match se dégrade, la traçabilité se perd et l'implémentation peut devenir **indéfinie**. On continue à travailler, mais on ne maîtrise plus vraiment ce qu'on est en train de construire.

> ⛔ La cérémonie d'implémentation ne doit pas devenir un espace d'ajout permanent. Sinon, l'équipe entre dans une boucle sans fin où elle implémente toujours plus, sans jamais refermer correctement ce qui avait été décidé.

🔁 Boucle de rétroaction vers ÉLICITER

Le comportement attendu n'est pas d'absorber silencieusement l'écart dans l'implémentation. Si un ajustement, une différence importante, un besoin additionnel ou un problème structurel apparaît, il faut **ramener les métriques et les signaux à ÉLICITER**.

Autrement dit :

- on **ne rajoute pas automatiquement** dans le scope en cours ;
- on **ne corrige pas la spécification de facto** en modifiant simplement le code ;
- on **capture** les observations, signaux, métriques, écarts et impacts ;
- on **retourne à ÉLICITER** si le besoin réel a changé ou si une nouvelle compréhension émerge ;
- on produit, au besoin, une **nouvelle version** qui repassera par les phases appropriées.

Cela reste vrai même si l'équipe peut techniquement régler le point immédiatement. Le problème n'est pas la capacité de le faire. Le problème est la perte de contrôle du système de travail si chaque découverte devient automatiquement un ajout d'implémentation.

Signal découvert en implémentation	Réaction attendue	Ce qu'on évite
Spécification qui ne couvre pas un cas important	Remonter le signal vers ÉLICITER ou DÉFINIR selon la nature de l'écart.	Inventer la règle en codant.
Nouvelle idée d'amélioration	Traiter comme un nouveau besoin ou une nouvelle	L'ajouter discrètement au scope courant.

	version potentielle.	
Écart majeur entre la réalité observée et le besoin initial	Ramener les métriques, faits et impacts à ÉLICITER.	Réécrire la cible en direct pendant le build.
Problème structurel découvert	Documenter, qualifier et faire réévaluer le besoin ou la stratégie.	Empiler des contournements non alignés.

🧮 Le travail effectué n'est pas perdu

Quand l'équipe doit arrêter, recadrer ou retourner à ÉLICITER, il ne faut surtout pas considérer que le travail déjà réalisé est du travail perdu. Au contraire, ce travail a servi à **découvrir**, à **révéler** et à **qualifier** un écart réel.

Il doit donc être comptabilisé comme du **travail contribuant à la découverte**, même s'il ne se traduit pas immédiatement par une livraison telle quelle. Ce travail produit de la connaissance utile :

- signaux supplémentaires ;
- métriques observées ;
- meilleure compréhension du problème ;
- confirmation d'une hypothèse ;
- invalidation d'une définition ;
- données concrètes pour une nouvelle version plus juste.

> ✅ Arrêter une implémentation pour revenir à ÉLICITER n'est pas un échec. C'est une démonstration de maîtrise du système et de protection de la qualité du plan de match.

✅ Critères d'une bonne implémentation

- Le travail exécuté correspond à ce qui a passé la DoR.
- Le code, les tests et les validations concordent avec les spécifications.
- Les validations nécessaires aux différents niveaux, incluant les intégrations pertinentes, sont réalisées.
- Les écarts découverts sont traités comme des signaux, pas comme des occasions d'étendre le scope librement.
- La traçabilité reste intacte entre besoin, définition, planification et exécution.
- L'équipe garde le contrôle du flux au lieu de subir une dérive continue des demandes.

⚠️ Pièges courants à éviter

Piège	Pourquoi c'est un problème	Comment l'éviter
Commencer sans vraie DoR	L'implémentation absorbe de la découverte non terminée.	Refuser l'entrée tant que les prérequis ne sont pas réellement satisfaits.
Traiter l'implémentation comme un espace de créativité produit	Le scope dérive et la concordance disparaît.	Réserver les changements de besoin aux phases amont.
Ajouter "tant qu'on y est"	Le travail devient imprévisible et potentiellement infini.	Capturer l'idée et la faire repasser par ÉLICITER au besoin.
Confondre validations d'intégration et diffusion réelle	On mélange build conforme et exposition réelle.	Garder DIFFUSER comme phase distincte de montage et de circulation réelle.
Considérer le retour en amont comme un échec	L'équipe cache les signaux au lieu de protéger le système.	Reconnaître le travail de découverte produit pendant l'implémentation.
Rester trop flexible rendu ici	Le système perd sa discipline au moment où l'exécution devrait être maîtrisée.	Accepter une agilité plus cadrée en implémentation que dans les phases amont.

➡️ Quand la cérémonie est réussie

La cérémonie d'implémentation est réussie lorsque l'équipe a réellement **buildé** ce qui avait été préparé, en conservant la maîtrise du scope, la concordance avec les spécifications et la qualité des validations. Le résultat attendu est un incrément suffisamment prouvé pour pouvoir ensuite entrer dans **DIFFUSER**, où l'on parlera davantage d'assemblage, de montage, d'intégration finale, de diffusion et de réalité opérationnelle.

> On peut rester agile ici, mais on ne doit plus être flou. La fluidité en implémentation vient de la qualité du travail fait en amont — et de la discipline à ne pas redéfinir la cible en plein build.

📝 Historique et traces des sessions

Les traces des rencontres réelles d'implémentation peuvent être conservées comme **pages enfants** sous cette cérémonie. L'objectif n'est pas d'alourdir la cérémonie, mais de garder une mémoire utile des décisions, écarts, validations et signaux observés pendant le build.

Gabarit recommandé pour une session :

- **Date et contexte** — sprint, lot, feature, correctif ou flux kanban concerné.
- **Participants** — rôles présents et personnes impliquées.
- **Stories / spécifications couvertes** — liens vers les éléments Jira, spécifications IF… THEN… et artefacts pertinents.
- **Points validés** — preuves de conformité, tests exécutés, validations techniques ou QA réalisées.
- **Écarts et signaux observés** — ambiguïtés, risques, défauts, hypothèses invalidées ou besoins émergents.
- **Actions et décisions** — action, propriétaire, échéance et suivi attendu.
- **Retours vers les phases appropriées** — ÉLICITER, DÉFINIR, PLANIFIER, DIFFUSER ou CONFIRMER selon la nature du signal.
- **Conclusion** — état de sortie : prêt pour DIFFUSER, à compléter, bloqué ou à recadrer.

📚 Références

- Comment implémenter — La référence complète de la phase IMPLÉMENTER.
- Cérémonie de planification — La cérémonie précédente qui prépare l'entrée en implémentation.
- Cérémonie de diffusion — La cérémonie suivante qui encadre le passage vers la diffusion réelle.
- Comment diffuser — La phase suivante, centrée sur l'assemblage, la diffusion et la réalité opérationnelle.
- SDLC officiel OPAL-RT — Le cycle de vie global dans lequel s'inscrit IMPLÉMENTER.
- Contrat qualité officiel OPAL-RT — Les attentes transversales de qualité, de traçabilité et de preuves.
- RACI officiel OPAL-RT — Les rôles et responsabilités de référence.
- Qualimétrie officielle OPAL-RT — La lecture organisationnelle des signaux et métriques qualité.
- Lexique / Terminologies — Les définitions officielles des termes utilisés.